

Fundamentos de Sistemas Robóticos com ROS 2

Etapa 4

Professor Mario Pego

mario.peggo@prof.infnet.edu.br



Cronograma



Fundamentos de Sistemas Robóticos com ROS 2

<i>Etapas</i>	<i>Segunda-feira/Quarta-feira</i>
Etapa 1	20/07/26 e 22/07/26
Etapa 2/Leitura TP1	27/07/26 e 29/07/26
Etapa 3	03/08/26 e 05/08/26
Etapa 4/Entrega do TP1/Leitura TP2	10/08/26 e 12/08/2026
Etapa 5	17/08/26 e 19/08/26
Etapa 6/Entrega TP2/Leitura do TP3	24/08/26 e 26/08/26
Etapa 7	31/08/26 e 02/09/26
Etapa 8/Entrega TP3	07/09/26 (FERIADO) e 09/09/26
Etapa 9/Leitura do AT	14/09/26 e 16/09/26
Entrega Assessment	21/09/26 e 23/09/26
Reentrega	28/09/26 e 30/09/26

Implementar comunicação assíncrona por tópicos com Publishers e Subscribers

O que vamos ver

1

Publishers em rclpy

create_publisher, timers e frequência

2

Subscribers em rclpy

create_subscription e callbacks

3

Mensagens geometry_msgs

A estrutura do Twist

4

QoS

Reliability, durability e profiles



rclpy

A biblioteca Python do ROS 2

ROS 2

rclpy (ROS Client Library for Python) é a biblioteca cliente do ROS 2 para Python.

Ela fornece a API que permite criar nós, publicar e receber mensagens, oferecer serviços, utilizar ações, parâmetros, temporizadores e demais recursos do ROS 2.

ARQUITETURA DO ROS 2

Sua aplicação Python
(usando rclpy)



rclpy (API Python)

rcl (ROS Client Library)

rmw (Middleware)

DDS (Fast DDS, Cyclone DDS, ...)



O QUE O rclpy PERMITE FAZER?



Criar nós
(Node)



Criar publishers



Criar subscribers



Criar services



Criar clients



Criar actions



Criar timers



Trabalhar com
parâmetros



Utilizar QoS



Executar múltiplos
callbacks

EXEMPLO BÁSICO

```
1 import rclpy
2 from rclpy.node import Node
3
4 class MeuNo(Node):
5     def __init__(self):
6         super().__init__('meu_no')
7         self.get_logger().info("Olá ROS 2!")
8
9 def main():
10     rclpy.init()
11     node = MeuNo()
12     rclpy.spin(node)
13     node.destroy_node()
14     rclpy.shutdown()
15
16 if __name__ == '__main__':
17     main()
```

Este código cria
um nó simples que
apenas imprime
uma mensagem no
log do ROS 2.

FUNÇÕES MAIS UTILIZADAS

Função	Finalidade
<code>rclpy.init()</code>	Inicializa o ROS 2
<code>Node()</code>	Cria um nó
<code>create_publisher()</code>	Cria um publicador
<code>create_subscription()</code>	Cria um assinante
<code>create_service()</code>	Cria um serviço
<code>create_client()</code>	Cria um cliente
<code>create_action_server()/create_action_client()</code>	Cria servidores/clients de ações
<code>create_timer()</code>	Cria um temporizador
<code>rclpy.spin()</code>	Mantém o nó em execução
<code>destroy_node()</code>	Destrói o nó
<code>rclpy.shutdown()</code>	Finaliza o ROS 2

RELAÇÃO COM C++

O ROS 2 possui duas bibliotecas clientes principais:



rclpy
Python



rclcpp
C++

A diferença está apenas na linguagem utilizada.
Ambas acessam a mesma camada `rcl` e o mesmo middleware DDS.

FLUXO DE EXECUÇÃO TÍPICO





Resumo: `rclpy` é a biblioteca oficial do ROS 2 para Python. Ela fornece todas as funcionalidades necessárias para desenvolver aplicações robóticas, permitindo criar nós, publicar e receber mensagens, utilizar serviços, ações, temporizadores e outros recursos do ecossistema ROS 2 por meio de uma API simples e integrada ao middleware DDS.

Relembrando: tópicos

Da Etapa 3 — comunicação assíncrona muitos-para-muitos

- Um publisher envia mensagens para um canal nomeado, sem saber quem está ouvindo
- Um ou mais subscribers recebem cada mensagem, de forma assíncrona
- Hoje vamos implementar os dois lados em Python com rclpy — os mesmos que você vai entregar no TP2
- O par Publisher/Subscriber é a base de praticamente todo sensor e atuador em ROS 2

Anatomia de um Publisher

`create_publisher(tipo, tópico, fila)`

salvar em `comunicacao_ros2/comunicacao_ros2/velocidade_pub.py`

```
from geometry_msgs.msg import Twist

class VelocidadePub(Node):
    def __init__(self):
        super().__init__(
            'velocidade_pub')
        self.pub = self.create_publisher(
            Twist, '/cmd_vel', 10)
```

Os 3 argumentos

Tipo da mensagem, nome do tópico e tamanho da fila de histórico (queue size) — exatamente a assinatura que você vai usar no `velocidade_pub.py` do TP2.

Exemplo completo: velocidade_pub.py

Publica Twist em /cmd_vel a 2 Hz — robô girando no próprio eixo

salvar em `comunicacao_ros2/comunicacao_ros2/velocidade_pub.py`

```
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist
class VelocidadePub(Node):
    def __init__(self):
        super().__init__('velocidade_pub')
        self.pub = self.create_publisher(Twist, '/cmd_vel', 10)
        self.create_timer(0.5, self.tick) # 2 Hz
    def tick(self):
        msg = Twist()
        msg.linear.x = 0.0
        msg.angular.z = 0.5
        self.pub.publish(msg)
def main(args=None):
    rclpy.init(args=args)
    rclpy.spin(VelocidadePub())
    rclpy.shutdown()
```

Timers e frequência de publicação

O timer é quem controla o ritmo, não um loop manual

- `create_timer(período_em_segundos, callback)` chama o callback automaticamente nesse intervalo
- Para 2 Hz (o que o TP2 pede), o período é $1/2 = 0.5$ segundos
- Nunca use `time.sleep()` dentro do callback — isso bloqueia todo o executor
- `'ros2 topic hz /cmd_vel'` deve indicar frequência próxima de 2.0 Hz — é assim que o TP2 valida esse requisito

Fórmula rápida

$\text{período (s)} = 1 / \text{frequência (Hz)}$

2 Hz $\rightarrow$ 0.5s (o valor do TP2)

10 Hz $\rightarrow$ 0.1s

0.2 Hz $\rightarrow$ 5s

geometry_msgs/msg/Twist

A mensagem usada no `/cmd_vel` do TP2

linear
x, y, z (m/s)

angular
x, y, z (rad/s)

No TP2: `linear.x = 0.0` (parado) e `angular.z = 0.5` (girando no próprio eixo) — só esses dois campos são usados.

Construindo a mensagem do velocidade_pub

TP2 Exercício 1

```
comunicacao_ros2/comunicacao_ros2/velocidade_pub.py
```

```
msg = Twist()  
msg.linear.x = 0.0    # sem deslocamento  
msg.angular.z = 0.5   # rad/s, giro no eixo  
  
self.pub.publish(msg)
```

Validação do TP2

'ros2 topic echo /cmd_vel' deve mostrar
linear.x: 0.0 e angular.z: 0.5 em cada
mensagem publicada.

Anatomia de um Subscriber

`create_subscription(tipo, tópico, callback, qos)`

`comunicacao_ros2/comunicacao_ros2/monitor_velocidade.py`

```
class MonitorVelocidade(Node):  
    def __init__(self):  
        super().__init__(  
            'monitor_velocidade')  
        self.create_subscription(  
            Twist, '/cmd_vel',  
            self.callback, qos)  
  
    def callback(self, msg):  
        ...
```

Nome real do TP2

`monitor_velocidade.py` é exatamente o Subscriber pedido no Exercício 2 — o 4º argumento aqui já é o `QoSProfile` `BEST_EFFORT`, que veremos a seguir.

Exemplo completo: monitor_velocidade.py

QoS BEST_EFFORT · calcula a norma da velocidade linear e o sentido de rotação

salvar em `comunicacao_ros2/comunicacao_ros2/monitor_velocidade.py`

```
import math
import rclpy
from rclpy.node import Node
from rclpy.qos import QoSProfile, ReliabilityPolicy
from geometry_msgs.msg import Twist
class MonitorVelocidade(Node):
    def __init__(self):
        super().__init__('monitor_velocidade')
        qos = QoSProfile(depth=10, reliability=ReliabilityPolicy.BEST_EFFORT)
        self.create_subscription(Twist, '/cmd_vel', self.callback, qos)
    def callback(self, msg):
        v = math.sqrt(msg.linear.x**2 + msg.linear.y**2 + msg.linear.z**2)
        sentido = 'horário' if msg.angular.z < 0 else 'anti-horário'
        self.get_logger().info(f'v={v:.2f} m/s, sentido={sentido}')
def main(args=None):
    rclpy.init(args=args)
    rclpy.spin(MonitorVelocidade())
    rclpy.shutdown()
```

O que evitar dentro de um callback

- Não bloqueie: nada de `time.sleep()`, `input()` ou chamadas de rede síncronas demoradas
- Não chame `rcipy.spin()` de novo dentro do callback
- Cálculos simples são seguros; processamento pesado deve rodar em outra thread
- Se precisar chamar um serviço a partir de um callback, use `call_async` — nunca a chamada bloqueante

O que é QoS e por que importa

Quality of Service — o contrato de confiabilidade da comunicação

- Define o comportamento da comunicação sob perda de pacotes, reconexões e diferenças de velocidade
- Publisher e subscriber negociam QoS na descoberta — perfis incompatíveis impedem a conexão
- O TP2 pede explicitamente BEST_EFFORT no monitor_velocidade — vamos entender por quê
- Configurado via QoSProfile, passado no create_publisher/create_subscription

Analogia

É como escolher entre carta registrada (garantida, mas lenta) e um anúncio de rádio (rápido, mas pode não chegar) — a escolha depende do que está sendo transmitido.

Reliability: RELIABLE vs BEST_EFFORT

Critério	RELIABLE	BEST_EFFORT
Garantia de entrega	Sim, reenvia se perder	Não, descarta silenciosamente
Latência	Maior (por causa do reenvio)	Menor
Cenário robótico ideal	Comandos de velocidade, serviços	Streaming de câmera/lidar a alta taxa
Usado no TP2	—	monitor_velocidade.py (Exercício 2)

Durability: VOLATILE vs TRANSIENT_LOCAL

Critério	VOLATILE	TRANSIENT_LOCAL
Mensagens antigas	Descartadas — só recebe o que chegar depois	Guarda as últimas N para novos subscribers
Cenário robótico ideal	Dados que mudam constantemente (odometria)	Mapas ou configurações publicadas 1x
Exemplo prático	/cmd_vel (o padrão usado no TP2)	/map em navegação (nav2)

History e Depth

Quantas mensagens ficam na fila de cada lado

- History KEEP_LAST(n): guarda só as últimas n mensagens (mais comum)
- History KEEP_ALL: guarda todas — cuidado com uso de memória
- Depth define o valor de n quando KEEP_LAST é usado
- No monitor_velocidade usamos depth=10 — o mesmo valor padrão do create_publisher do velocidade_pub

Valor típico

depth=10 é um bom padrão para tópicos de controle como /cmd_vel. Sensores de alta frequência (câmera, lidar) costumam usar depth menor (1-5).

Profiles pré-definidos

Atalhos para combinações comuns

```
from rclpy.qos import (
    qos_profile_sensor_data,
    qos_profile_system_default,
    qos_profile_services_default,
)

self.create_subscription(
    Image, 'camera/image',
    self.cb, qos_profile_sensor_data)
```

Por que o TP2 pede QoS manual

sensor_data já vem BEST_EFFORT pronto, mas o TP2 pede para você montar o QoSProfile manualmente no monitor_velocidade — é assim que se aprende o que cada campo realmente faz.

Configurando o QoSProfile do monitor_velocidade

Exatamente o objeto usado no Exercício 2 do TP2

```
comunicacao_ros2/comunicacao_ros2/monitor_velocidade.py
```

```
from rclpy.qos import QoSProfile, ReliabilityPolicy

qos = QoSProfile(
    reliability=ReliabilityPolicy.BEST_EFFORT,
    depth=10,
)

self.create_subscription(Twist, '/cmd_vel', self.callback, qos)
```

Registrando o `entry_point`

`monitor_velocidade` também precisa aparecer no `setup.py`

salvar em `comunicacao_ros2/setup.py`

```
entry_points={
    'console_scripts': [
        'velocidade_pub = '
        'comunicacao_ros2.velocidade_pub:main',
        'monitor_velocidade = '
        'comunicacao_ros2.monitor_velocidade:main',
    ],
},
```

Não esqueça

`velocidade_pub` já estava registrado desde a Etapa 3. Agora acrescente a linha de `monitor_velocidade` — sem isso, `'ros2 run comunicacao_ros2 monitor_velocidade'` retorna `'No executable found'`.

Recompilando e testando o par

velocidade_pub foi alterado e monitor_velocidade é novo — os dois exigem build

Terminal 1 — build + velocidade_pub

```
$ cd ~/tp1_ws
$ colcon build --packages-select \
  comunicacao_ros2
$ source ~/tp1_ws/install/setup.bash
$ ros2 run comunicacao_ros2 \
  velocidade_pub
```

Terminal 2 — monitor_velocidade

```
$ source ~/tp1_ws/install/setup.bash
$ ros2 run comunicacao_ros2 \
  monitor_velocidade

[INFO] v=0.00 m/s, sentido=anti-horário
[INFO] v=0.00 m/s, sentido=anti-horário
```

Por que recompilar

colcon build precisa rodar de novo sempre que um arquivo .py muda OU um novo entry_point é registrado — mesmo que o pacote já exista. Sem isso, o executor continua usando a versão antiga instalada.

Cenários robóticos x política de QoS

Cenário	Reliability	Durability
/cmd_vel (velocidade_pub / monitor_velocidade)	BEST_EFFORT (no subscriber)	VOLATILE
Imagem de câmera a 30fps	BEST_EFFORT	VOLATILE
Mapa publicado uma vez (nav2)	RELIABLE	TRANSIENT_LOCAL
Estado da bateria	RELIABLE	VOLATILE

Compatibilidade de QoS

O que acontece quando publisher e subscriber divergem

- RELIABLE (publisher) + BEST Effort (subscriber) → funciona (o subscriber aceita menos garantia)
- É exatamente o caso do TP2: velocidade_pub usa o padrão RELIABLE, monitor_velocidade pede BEST Effort — compatível
- BEST Effort (publisher) + RELIABLE (subscriber) → INCOMPATÍVEL, a conexão não é estabelecida
- O ROS 2 não avisa com um erro explícito — o sintoma é 'não chegam mensagens'

Como depurar

`ros2 topic info /cmd_vel --verbose` mostra o QoS de cada publisher e subscriber conectado — use antes de entregar o TP2 para confirmar que os dois nós estão compatíveis.

Inspecionando QoS na prática

ros2 topic info --verbose — o comando de validação do TP2

```
$ ros2 topic info /cmd_vel --verbose
Type: geometry_msgs/msg/Twist
Publisher count: 1
  Node name: velocidade_pub
  QoS profile:
    Reliability: RELIABLE
    Durability: VOLATILE
    History (Depth): KEEP_LAST (10)
```

No TP2

O Exercício 4 pede exatamente essa leitura, junto com 'ros2 topic hz /cmd_vel' — saber interpretar cada campo aqui é parte da validação e do relatório técnico.

Competência 1

OK

`velocidade_pub.py`

Publisher Twist em /cmd_vel a 2 Hz, pronto para o TP2

OK

`monitor_velocidade.py`

Subscriber com QoS BEST_EFFORT, norma e sentido de rotação

OK

`RELIABLE / BEST_EFFORT /
TRANSIENT_LOCAL`

sei escolher QoS para cada cenário

OK

`ros2 topic info --verbose / hz`

sei validar frequência e QoS de um tópico ativo

Próximo passo

Com `velocidade_pub.py` e `monitor_velocidade.py` prontos, os Exercícios 1 e 2 do TP2 já estão cobertos. Na próxima competência, vamos implementar `modo_server.py` e `modo_client.py` — os serviços Client/Server do Exercício 3.

Implementar chamadas
síncronas entre nós com
serviços Client/Server ROS 2

O que vamos ver

1

Interfaces de serviço

std_srvs e o padrão request/response

2

Server em rclpy

create_service e callbacks

3

Client em rclpy

create_client, call_async e Future

4

Executors e boas práticas

Evitando deadlocks e timeouts

Relembrando: serviços

Da Etapa 3 — comunicação síncrona um-para-um

- Um client envia uma requisição e aguarda uma resposta de um server
- Diferente do tópico, existe confirmação: o client sabe que a operação foi processada
- Hoje vamos implementar `modo_server.py` e `modo_client.py` — (Exercício 3 do TP2)

Serviço em vez de tópico?

Revisão prática da decisão

- Use serviço quando a operação é pontual e você precisa de confirmação (ligar um modo, resetar um estado)
- Use tópico quando o dado é contínuo e perdas ocasionais são aceitáveis (sensores, cmd_vel)
- Serviço bloqueia o client até responder — não use para operações demoradas (para isso existe Ação)

Exemplo do TP2

Ligar/desligar o 'modo autônomo' de um robô é uma operação pontual com confirmação — por isso o TP2 pede um serviço (modo_server), não um tópico.

std_srvs: tipos prontos

SetBool é o tipo usado pelo modo_server do TP2

Tipo	Request	Response
<code>std_srvs/srv/Empty</code>	(vazio)	(vazio)
<code>std_srvs/srv/Trigger</code>	(vazio)	bool success, string message
<code>std_srvs/srv/SetBool</code>	bool data	bool success, string message

Anatomia do SetBool.srv

A interface usada no modo_server do TP2

```
# SetBool.srv
bool data
---
bool success
string message
```

Como funciona

O gerador de interfaces do ROS 2 lê esse arquivo e cria SetBool.Request (com o campo data) e SetBool.Response (com success e message) — é isso que você importa em modo_server.py.

Anatomia de um Server

`create_service(tipo, nome, callback)`

salvar em `comunicacao_ros2/comunicacao_ros2/modo_server.py`

```
from std_srvs.srv import SetBool

class ModoServer(Node):
    def __init__(self):
        super().__init__(
            'modo_server')
        self.create_service(
            SetBool, 'modo',
            self.callback)
```

O callback

Recebe (request, response) e DEVE retornar o objeto response preenchido. request.data é o bool True/False enviado pelo modo_client.

Exemplo completo: modo_server.py

True → modo autônomo · False → modo manual

salvar em `comunicacao_ros2/comunicacao_ros2/modo_server.py`

```
import rclpy
from rclpy.node import Node
from std_srvs.srv import SetBool
class ModoServer(Node):
    def __init__(self):
        super().__init__('modo_server')
        self.create_service(SetBool, 'modo', self.callback)
    def callback(self, request, response):
        if request.data:
            response.message = 'Robô em modo autônomo ativado'
        else:
            response.message = 'Robô em modo manual ativado'
        response.success = True
        return response
def main(args=None):
    rclpy.init(args=args)
    rclpy.spin(ModoServer())
    rclpy.shutdown()
```

Campos de resposta: success e message

O que o modo_client vai ler

- success=True/False informa objetivamente se a operação funcionou
- message é exatamente o texto que o TP2 pede: 'Robô em modo autônomo ativado' ou 'Robô em modo manual ativado'
- O client SEMPRE deve checar response.success antes de assumir sucesso
- Evite lançar exceções não tratadas dentro do callback — isso derruba o server

Validação do TP2

'ros2 run comunicacao_ros2 modo_client -- true' deve imprimir exatamente 'Robô em modo autônomo ativado' — a string precisa bater com o que o Server devolve.

Anatomia de um Client

create_client + wait_for_service

salvar em `comunicacao_ros2/comunicacao_ros2/modo_client.py`

```
from std_srvs.srv import SetBool

class ModoClient(Node):
    def __init__(self):
        super().__init__(
            'modo_client')
        self.cli = self.create_client(
            SetBool, 'modo')
        while not self.cli.wait_for_service(
            timeout_sec=1.0):
            self.get_logger().info('aguardando...')
```

wait_for_service

Bloqueia até o modo_server aparecer no grafo (ou até o timeout). Sem isso, rodar o client antes do server resultaria em erro.

Dois padrões de chamada

Padrão	Como funciona	Quando usar
call() (síncrono)	Bloqueia a thread até a resposta chegar	Scripts simples, fora de um callback
call_async() (assíncrono)	Retorna um Future imediatamente	Dentro de nós com timers/callbacks ativos

call_async() e Future

Como o modo_client envia a requisição

salvar em `comunicacao_ros2/comunicacao_ros2/modo_client.py`

```
request = SetBool.Request()
request.data = True
future = self.cli.call_async(request)

# future.done() == False agora
# ... o executor processa em background ...
# future.result() -> SetBool.Response
```

Future

É um objeto padrão de programação assíncrona: representa um resultado que ainda não existe, mas vai existir. `call_async()` nunca bloqueia.

Aguardando a resposta

`spin_until_future_complete` — usado no `modo_client.py`

salvar em `comunicacao_ros2/comunicacao_ros2/modo_client.py`

```
def main(args=None):  
    rclpy.init(args=args)  
    client = ModoClient()  
    req = SetBool.Request()  
    req.data = sys.argv[1] == 'true'  
    future = client.cli.call_async(req)  
    rclpy.spin_until_future_complete(  
        client, future)  
    print(future.result().message)
```

`sys.argv`

O TP2 pede que o Client leia o argumento `true/false` da linha de comando — `sys.argv[1]` captura exatamente esse valor.

Exemplo completo: modo_client.py

Lê true/false de sys.argv e envia a requisição

salvar em `comunicacao_ros2/comunicacao_ros2/modo_client.py`

```
import sys
import rclpy
from rclpy.node import Node
from std_srvs.srv import SetBool

class ModoClient(Node):
    def __init__(self):
        super().__init__('modo_client')
        self.cli = self.create_client(SetBool, 'modo')
        while not self.cli.wait_for_service(timeout_sec=1.0):
            self.get_logger().info('aguardando modo_server...')

def main(args=None):
    rclpy.init(args=args)
    client = ModoClient()
    req = SetBool.Request()
    req.data = sys.argv[1].lower() == 'true'
    future = client.cli.call_async(req)
    rclpy.spin_until_future_complete(client, future)
    print(future.result().message)
    rclpy.shutdown()
```


Registrando os entry_points

modo_server e modo_client precisam entrar no setup.py

salvar em `comunicacao_ros2/setup.py`

```
entry_points={
    'console_scripts': [
        # ... velocidade_pub, monitor_velocidade
        'modo_server = '
        'comunicacao_ros2.modo_server:main',
        'modo_client = '
        'comunicacao_ros2.modo_client:main',
    ],
},
```

Não esqueça

São dois nomes novos para acrescentar à mesma lista de console_scripts. Sem os dois, nem 'ros2 run ... modo_server' nem 'ros2 run ... modo_client' funcionam.

Recompilando e subindo o Server

colcon build de novo — dois nós novos foram registrados

```
$ cd ~/tp1_ws  
$ colcon build --packages-select comunicacao_ros2  
$ source install/setup.bash  
$ ros2 run comunicacao_ros2 modo_server
```

```
[INFO] [modo_server]: nó iniciado, aguardando chamadas em /modo
```

Deixe rodando

Mantenha esse terminal aberto — modo_server precisa estar ativo para os testes com 'ros2 service call' e modo_client a seguir.

Executors: Single vs MultiThreaded

Quem decide a ordem em que os callbacks rodam

- `SingleThreadedExecutor` (padrão): processa um callback por vez, em sequência
- `MultiThreadedExecutor`: processa múltiplos callbacks em paralelo, em threads diferentes
- `modo_server` e `modo_client`, do jeito que construímos, funcionam bem com o executor padrão
- `MultiThreaded` só é necessário quando um mesmo nó precisa atender E chamar serviços ao mesmo tempo

Regra prática

Se seu nó só publica/assina tópicos ou só atende serviços (não os dois cruzados), o executor padrão single-threaded é suficiente.

Por que `call_async` pode travar

O problema do deadlock em executor single-threaded



Se o Callback A usa `call()` (síncrono/bloqueante) e espera pela resposta do Serviço B, mas o executor single-threaded só tem UMA thread — a mesma thread que precisaria processar a resposta de B está travada esperando dentro de A. Resultado: deadlock permanente.

Solução: use `call_async()` dentro de callbacks (como fizemos no `modo_client`), ou um `MultiThreadedExecutor`.

Timeouts e serviço indisponível

Nunca espere para sempre

```
if not self.cli.wait_for_service(  
    timeout_sec=3.0):  
    self.get_logger().error(  
        'modo_server indisponível')  
    return  
  
future = self.cli.call_async(req)
```

Por que importa

Sem timeout, o modo_client pode ficar preso indefinidamente se o modo_server não estiver rodando. Sempre trate esse caso no seu relatório do TP2.

Inspecionando o serviço

ros2 service list — o comando de validação do TP2

```
$ ros2 service list  
/modo_server/describe_parameters  
/modo
```

```
$ ros2 service type /modo  
std_srvs/srv/SetBool
```

No TP2

'ros2 service list' com o Server ativo deve exibir o serviço 'modo' registrado — é exatamente o critério de validação do Exercício 3.

Testando modo_server e modo_client

Duas formas de chamar o serviço

ros2 service call

```
$ ros2 service call /modo \
  std_srvs/srv/SetBool 'data: true'

response:
success=True
message='Robô em modo
  autônomo ativado'
```

modo_client (TP2)

```
$ source ~/tp1_ws/install/setup.bash
$ ros2 run comunicacao_ros2 modo_client --
true
Robô em modo autônomo ativado

$ ... modo_client -- false
Robô em modo manual ativado
```

Dica

Teste primeiro com 'ros2 service call' para isolar se o problema está no Server — só depois valide pelo modo_client.py.

Competência 2

OK

`modo_server.py`

Server SetBool com as duas respostas exatas do TP2

OK

`modo_client.py`

Client assíncrono lendo true/false de sys.argv

OK

`spin_until_future_complete`

Aguardar e ler o resultado de uma chamada

OK

`ros2 service list / call`

Inspeccionar e testar o serviço 'modo' pela CLI

Você está pronto para o TP2

Com `velocidade_pub.py`, `monitor_velocidade.py`, `modo_server.py` e `modo_client.py` implementados nesta etapa, os Exercícios 1, 2 e 3 do TP2 já estão cobertos. O Exercício 4 (isolamento por `ROS_DOMAIN_ID=42`, namespace `/robo1/cmd_vel` e os 5 comandos de inspeção via CLI) já foi demonstrado por completo na Etapa 3, Competência 1 — incluindo a falha e a correção do isolamento em dois terminais. Revise essa competência antes de entregar o TP2.